Axona

A Learning-Adaptive Distributed Hash Table

David A. Smith
Axona.net

An Axona Explainer v0.4.27 2026-05-30

The technology is shaped by the mission.

Neurons that fire together, wire together.
Donald Hebb · 1949

I The Mission

Axona is the engineering substrate for a privacy-first decentralized internet. The protocol is designed for a specific job: to carry the routing and publish/subscribe workload of that internet, on consumer devices, in real browsers, against the actual physics of the network it runs on. Everything else about the design—the architecture, the algorithms, the simulator, the deployment path—is shaped by that job.

The job is hard for an unfamiliar reason. The internet of 2026 has no shortage of clever distributed systems, but almost all of them are *distributed underneath, central on top*: a constellation of servers that the user reaches through a single corporate gateway.[1] A real peer-to-peer internet—one where two strangers can find each other, exchange a message, and route around an outage without asking permission of any intermediary—requires a different kind of substrate. The substrate must be *federation-free at every layer*: addressing, routing, discovery, group communication, eventual delivery under churn.

[1] The pattern is so common we have stopped noticing it. CDNs, federated identity providers, cloud auth, app stores, the "serverless" frameworks that run on three or four hyperscaler clouds—all examples of distribution-as-implementation-detail sitting under a single point of decision.

A distributed hash table is the addressing layer of that substrate. Without one, every other peer-to-peer primitive collapses back into a centralized service: there is no way to find the user you want to talk to, no way to discover the file you want to download, no way to subscribe to the channel you want to follow, without somebody telling you where it lives. With one, those operations are mechanical: the network itself routes you to the right peer.

The mission is to build the addressing layer that a real peer-to-peer internet needs. **Axona** is what that addressing layer looks like.

II The Problem

Imagine you and a million strangers each hold one piece of a giant jigsaw puzzle. Someone walks up and asks for piece #438,291. How do you find it?

Option 1: Have a central directory. Some company keeps a giant list: "piece 438,291 is held by Alice." Easy to look up—but now that company controls everything. They can shut you down, spy on you, charge you money, or just go bankrupt and take the directory with them.

Option 2: Give every piece an "address," and design a system where the network *itself* knows how to route requests to the right person, with no central directory. This is what a **distributed hash table** (DHT) does.[2]

The trick: how do you find the closest node when you don't know who's in the network and you only know a handful of peers?

The dominant DHT algorithm is called **Kademlia**, designed in 2002.[3] Every node has a 160-bit ID—a very large random number. To measure "distance" between two IDs, you XOR them, comparing bit by bit. To find a target, you ask the closest peer you know. They tell you about peers *they* know that are even closer. You ask one of those. Repeat.

The math is clean: each step at least *halves* the remaining distance, so you reach any target in about $\log_2(N)$ hops. With a million nodes, that's 20 steps. Provably correct, used in production all over the internet.

There's just one problem.

[2] DHTs are the machinery behind BitTorrent, IPFS, parts of Ethereum, and the hidden services on Tor. Every node gets a random ID; every piece of content gets an ID; lookups route from one to the other.

[3] Maymounkov & Mazières, "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric," *Proc. IPTPS 2002*.
XOR is a bitwise operation that returns 1 wherever two bits differ and 0 wherever they match. Two IDs that share many leading bits XOR to a small number; two unrelated IDs XOR to a large one.

III The Latency Tax

Twenty hops sounds fast, but each hop sends a message between two random computers somewhere on Earth. If your peers are scattered randomly across the globe, the average pair sits about half the planet apart—roughly 100 milliseconds round trip.

20 hops × 100 ms = **2 seconds.** To find something. Every time.

For real-time applications—voice calls, online games, live messaging, push notifications—2 seconds is unusable. The math says routing is logarithmic, which is fast; the physics says each step is slow because peers are far apart. That's the tension.

The protocol attacks the latency problem with two ideas, one of which is borrowed straight from neuroscience.

IV Putting the Address in the Address

The first idea is almost embarrassingly simple. It's called **G-DHT** (Geographic DHT).

Kademlia node IDs are random. A node in Tokyo and a node in Berlin have IDs with no relationship to where they actually are. That's *why* hops are random and slow—random IDs mean random geography.

G-DHT changes one thing: the **first 8 bits** of every node ID encode where the node says it is on Earth.

The trick uses Google's S2 library, which divides Earth's surface into cells along a curve called a **Hilbert curve**. Its useful property: places near each other on Earth get cell numbers near each other. So XORing two S2 cell numbers gives a small result for nearby places and a large one for distant places.

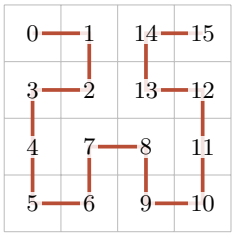
Hilbert curves are space-filling: they trace a single continuous path that visits every cell in a grid. Their useful property is that consecutive points on the curve are also adjacent in space.

Random IDs

7	13	3	10
1	15	9	4
6	0	12	8
5	11	2	14

neighbors in space, strangers in ID

Hilbert (S2) IDs



consecutive integers walk a connected path

Figure 1: Two arrangements of the integers 0–15 on a 4 × 4 grid. With random IDs, sequential numbers scatter unpredictably; cells that are spatial neighbors are numerical strangers. With S2's Hilbert-curve IDs, sequential numbers form a connected path—a continuous walk that never leaves a small neighborhood. XOR distance in ID space now approximates distance on the ground.

Now your node ID looks like: `[8-bit geographic cell][256-bit hash of public key]`—a 264-bit address space, encoded as 66 lowercase hex characters in the wire protocol.

The routing algorithm doesn't change at all, but XOR distance now approximately tracks physical distance. When a node looks for a "close" peer in ID space, it tends to find one that's also physically close. Local traffic stays local. The 20-hop world tour becomes a 13-hop journey around the neighborhood, with each hop maybe 7 ms instead of 100.

Total: about 91 ms instead of 2 seconds. **Roughly 20× faster** for regional traffic, just from changing the *structure* of the addresses.

The cost: nodes can lie about where they are. This is a "cooperative trust" assumption. Mitigations exist, but for now G-DHT trades some defense against location-spoofing for a large speedup.

Of course a user can choose to associate themselves with any location in the world. This very rough location is a kind of area code—but the address it is attached to is provably yours, independent of where you actually are, because the address is derived from your cryptographic key and proven the moment you connect. The section *Identity You Can't Fake*, after the pub/sub discussion, explains exactly how.

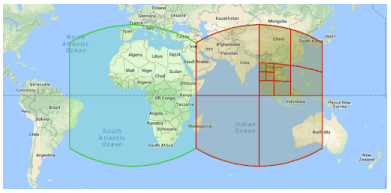


Figure 2: S2 cell decomposition of Earth's surface, projected through six cube faces along a Hilbert space-filling curve. Nearby points on the globe land at numerically close cell IDs; XOR distance in identifier space therefore approximates physical distance, for free.

An attacker who falsifies their geographic prefix degrades the locality benefit but not routing correctness. Mitigations exist—learned coordinates, geographic proof-of-work, IP–ASN binding—but G-DHT trades some defense against location-spoofing for a large speedup.

The next section addresses this directly. Axona, the neuromorphic protocol layered on top of G-DHT, treats *measured one-way latency* as a first-class signal in its scoring of every connection: a connection's vitality is the product of how often it's used and how recently, but it is gated by how cheap it actually is to send a message across.[4] A node that claims a cell prefix in Frankfurt while actually

[4] The Vitality Function on the next page makes this explicit; the per-hop latency probe is part of every routed lookup.

answering from São Paulo cannot fake the round-trip time. The first few lookups that try to use its synapses see latencies of 200+ ms when the prefix would predict 20. The cheating node's connections accumulate weight slowly and get out-competed by honest neighbors at every eviction decision. The cheat is not blocked—the address space allows the claim—but it is structurally penalized in proportion to how far the lie is from the truth.

V A Network That Learns Like a Brain

THE SECOND IDEA—neuromorphic routing—is the heart of the protocol, and what gives **Axona** its name. It asks a different question:

> *What if, instead of engineering shortcuts into the network, the network learned its own shortcuts based on which paths actually work?*

To explain how, we reach for a strange-but-perfect analogy: the human brain.

The Engineering Problem That Brains Already Solved

A single neuron in your brain connects to roughly 10,000 others through structures called **synapses**. There are about 86 billion neurons total. Any given neuron *could* connect to almost any other, but maintaining synapses is energetically expensive—so neurons are stuck with a hard cap, surrounded by far more potential partners than they can afford to keep.

How does the brain decide *which* connections to keep?

This is the same problem facing a peer in a peer-to-peer network. A web browser can only maintain about 50–100 simultaneous WebRTC connections. The network might have hundreds of thousands of nodes. Which 50 do you keep?

WebRTC is the technology that lets browsers talk directly to each other, without a server in the middle. Safari caps at about 95 simultaneous connections; Firefox at about 190.

The brain solved this problem hundreds of millions of years ago. The solution is called **long-term potentiation**, or LTP.

The Brain's Trick

In 1973, two scientists—Bliss and Lømo—ran a now-classic experiment.[5] They zapped a particular pathway in a rabbit's brain with high-frequency electrical pulses. Afterward, that pathway responded *more strongly* to subsequent signals—not for seconds, but for *weeks*. The connections had physically gotten stronger.

[5] Bliss & Lømo, "Long-lasting potentiation of synaptic transmission in the dentate area of the anaesthetized rabbit following stimulation of the perforant path," *J. Physiology* 232(2), 1973, pp. 331–356.

Decades of follow-up research filled in the details:

1. **The coincidence detector.** Synapses have a special kind of receptor (called NMDA) that only activates when *both* sides of the

connection fire at roughly the same time. It's not "I fired" or "you fired"—it's specifically "we fired together."

2. **The strengthening signal.** When that coincidence happens, calcium rushes in and triggers the synapse to insert *more* of a different kind of receptor (AMPA), making the connection physically more sensitive.

3. **Consolidation.** Over the next half hour or so, new proteins get made and new physical structures grow. The change becomes permanent.

4. **Decay.** Connections that *don't* get used regularly weaken over time, in a process called long-term depression (LTD).[6]

5. **A protective tag.** Recently strengthened synapses get a temporary "tag" that protects them from being overwritten by competing changes—but only for a brief window.[7]

Donald Hebb summarized this in 1949—before the molecular details were known—in the rule that's now bedrock in neuroscience and AI:[8]

Neurons that fire together, wire together.

Every artificial neural network you've ever heard of—every part of ChatGPT, every image recognition system, every recommendation algorithm—ultimately runs on a digital descendant of this rule.

[6] Stanton & Sejnowski, "Associative long-term depression in the hippocampus induced by Hebbian covariance," *Nature* 339(6221), 1989, pp. 215–218.

[7] Frey & Morris, "Synaptic tagging and the late phase of long-term potentiation," *Nature* 385(6616), 1997, pp. 533–536.
Without this tag, learning would be self-destructive: the synapses you just learned were useful would be the *first* ones overwritten by the next signal.

[8] Hebb, *The Organization of Behavior: A Neuropsychological Theory*, Wiley, 1949.

Translating Neurons into Network Routing

The claim is that the brain's mechanism translates *literally* to peer-to-peer routing—no metaphor needed.

The mapping from neurons to Axona's routing logic:

In the brain	In Axona
A synapse (neural connection)	A connection to a peer
Synaptic strength (weight 0 to 1)	A learned weight on each connection
LTP: co-firing strengthens	A successful lookup increments the weights along its path
LTD: disuse weakens	Every connection slowly decays over time
Synaptic tagging (protection)	Recently used connections are protected briefly
Pruning of unused connections	Lowest-scoring connection is evicted when a new one wants in

That is what Axona *is*: a routing system where every connection has a weight that goes up when used and decays when not. The network's "memory" is its routing table, and the routing table evolves into whatever shape best serves the actual traffic.

The Vitality Function

The core of Axona's neuromorphic routing is a single equation that scores every connection:

$$vitality \;=\; weight \;\times\; recency$$

The **weight** is a number between 0 and 1, updated like this:

- Every successful lookup that uses the connection raises its weight by 0.05 (capped at 1).
- Every "tick" of the clock multiplies every weight by 0.995—a slow decay.

The **recency** is 1.0 for the first 20 ticks after a connection is used—the protective tag from synaptic tagging—then drops off exponentially.

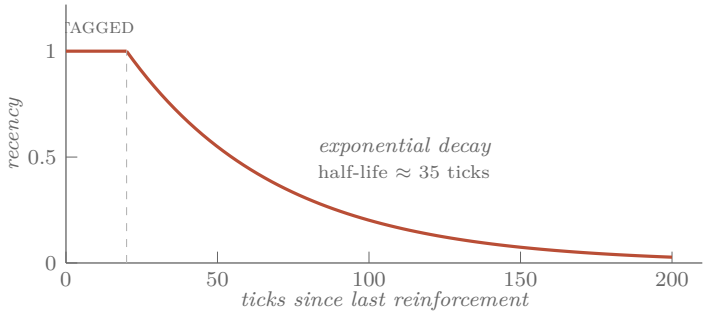


Figure 3: The recency multiplier in Axona's vitality function. For 20 ticks after a connection is used, recency stays locked at 1.0—the synaptic tag, protecting freshly learned shortcuts from being immediately overwritten. After that, it decays exponentially with a half-life of about 35 ticks. A connection unused for ∼150 ticks contributes essentially nothing to its vitality, regardless of accumulated weight.

When a new connection wants in but the routing table is full, the connection with the lowest vitality gets evicted. Frequently used connections accumulate weight and stay. Unused ones decay and get evicted. Recently strengthened ones are temporarily protected.

That's it. That's the core.

VI The Five Operations

Every behavior in Axona falls into one of five categories—and the categories mirror how *any* adaptive system works:

1. Navigate — pick the next hop for a lookup. Axona scores each candidate by combining XOR distance progress, learned weight, and observed latency. We call this combined score the candidate's **activation potential** (AP)—borrowing the neuroscience term for how strongly a neuron is driven to fire; the higher a connection's AP, the more likely it is to "fire" the lookup down that path. The latency factor halves the score every 100 ms—so a peer that's mathematically a *bit* further but physically a *lot* closer wins.

2. Learn — strengthen what works. Three mechanisms run on every successful lookup:

- **LTP**: every connection on a fast successful path gets stronger.
- **Hop caching**: every intermediate node along the path remembers the *destination*, not just the next hop. Next time, the path is shorter.
- **Triadic closure**: if you keep seeing peer A relay messages to peer C through you, you introduce them directly. The triangle "A–you–C" becomes a direct edge "A–C."

Named after social network theory: dense triangles emerge in any network shaped by interaction.

3. Forget — decay everything that isn't reinforced; evict by vitality.

4. Explore — inject occasional randomness. A purely greedy router would lock onto the first decent shortcut and never find better ones. Axona has two exploration tricks: a "temperature" that decays over time but spikes when something breaks (heat up when surprised, cool down when stable), and an "epsilon-greedy" rule that picks a random first hop 5% of the time, just to keep options alive.

5. Structure — bootstrap and maintain the basic shape of the network when new nodes join.

The template is universal: act, learn from feedback, forget the obsolete, occasionally try something new, maintain basic structure. This is how any good adaptive system has to work.

VII Axonal Pub/Sub

A REAL peer-to-peer network needs more than "find the node holding key X." It also needs **broadcast**: a publisher should be able to send a message to all subscribers of a topic, without knowing who they are.

This is publish/subscribe, or "pub/sub." Think of how YouTube notifies subscribers of a new video—except without YouTube being in the middle.

In neuroscience, the *output* of a neuron—the long branching cable that delivers signals to many downstream targets—is called an **axon**. Axona builds axonal delivery trees:

- A topic has an ID, derived from the publisher's location and the topic name.
- Subscribers send a message routed through the DHT toward that topic ID.

- The first node already participating in the topic's tree intercepts the subscriber and adds them as a child.
- When a publisher publishes, the message flows from the root down through all the branches.

WHY A TREE AT ALL is a scaling question. A topic with a few dozen subscribers doesn't really need one—a single relay node, or maybe two or three, can absorb every publish and fan it out directly. The tree only earns its complexity once a topic's audience grows. At hundreds, thousands, or tens of thousands of subscribers, no small set of relays can carry that much traffic: every publish would compete for the same outbound bandwidth at the root, and a browser peer's hard 50–190-connection WebRTC cap would put a ceiling on the audience well before then. So when a relay starts to overload, it **splits**: it picks one of its synaptome peers as a new **sub-axon** and hands that sub-axon a batch of its existing children. Future subscribers route to whichever live axon is now closest—increasingly a sub-axon, not the root. The tree therefore grows *in the direction of its audience*: branches sprout where subscribers cluster, and stay sparse where they don't. There is no architectural ceiling on subscriber count—the tree adds branches as needed.

That scaling is exactly what makes self-healing essential. A tree with a few nodes can ride out churn by luck—if the relay doesn't happen to die, nothing notices. A tree spanning thousands of nodes loses pieces *continuously*, simply because there are more nodes for churn to strike. The bigger the tree, the more surface area churn has to chew on. Self-healing is therefore not a feature added on top of the tree; it is the precondition that lets the tree get large at all.

Tree-building this way is standard pub/sub fabric. The useful property—the one that lets the tree get arbitrarily large without falling apart—is that **the tree heals itself with no special machinery**. Every member of the tree periodically re-issues its subscribe message. If the tree is intact, nothing changes. If a parent died, the re-subscribe naturally lands on whichever live ancestor is now closest. No heartbeats, no failure detection, no parent tracking. The same mechanism that *builds* the tree *repairs* it.

SELF-HEALING ALONE is not enough. A subtree that re-attaches in three seconds has still *missed* whatever messages flowed past the dead branch in those three seconds. Axona closes that gap with a second mechanism: every relay carries a small **replay cache**—the most recent N messages it has forwarded for each topic, typically the last 100.[9] Every re-subscribe carries the time of the last message

[9] The cache is bounded per topic per relay, not per subscriber, so the cost is the same whether a topic has ten subscribers or ten thousand.

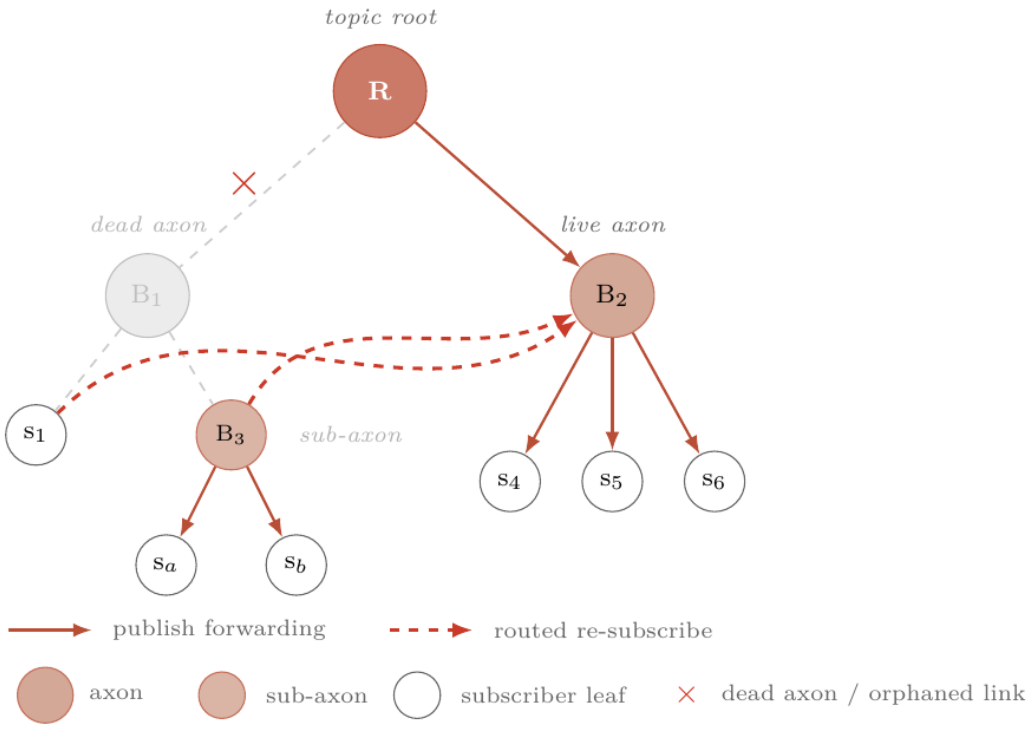


Figure 4. When a branch axon dies, its orphans heal the tree by re-issuing their subscribe. Here, branch B_1 has died and its link to the topic root R is broken. Two of B_1's children—a leaf subscriber s_1 and a sub-axon B_3—each issue a routed re-subscribe (dashed); both land on the surviving live axon B_2, which adopts them. Crucially, B_3's own subtree (s_a, s_b) does *not* need to re-subscribe individually: B_3 continues publishing to them throughout, and the whole subtree moves intact when B_3 re-attaches. Repair happens at the sub-axon level, not the leaf level—one subscribe per surviving subtree, not one per subscriber.

its sender actually saw—a field called `lastSeenTs`. When a relay receives a re-subscribe, it scans its cache for messages newer than that timestamp and replays them down the branch before the subscription resumes its steady-state forwarding.

The two mechanisms compose. Tree-healing keeps the topology correct in the face of churn; the replay cache makes the *message stream* correct in the face of the small windows the tree spends in transition. A subscriber whose parent dies, whose subtree migrates to a different live ancestor, and which then re-subscribes against that new ancestor receives, in order: the missed history from the replay cache, followed by the live stream from the new branch. The seam is invisible at the application layer.

Result: under 5% churn (5% of nodes joining and leaving constantly), Axona still delivers 100% of messages. After three refresh cycles, it recovers from any disruption.

Above the axonal tree sits a feed-style application layer with **four verbs**—`pub`, `sub`, `pull`, `metrics`. They cover what a real social or

agent-collaboration application asks of a substrate: author new content, attach to a topic, fetch a referenced post on demand, and let a publisher see verifiable reach across the cascade—without identifying any individual subscriber. (Out-of-band, the same peer exposes a small `send/notify/onMessage` surface for unicast, but that is the direct-messaging layer, not the feed.) Encryption, schema, and ordering belong to the application above this layer; the protocol carries opaque bytes.

VIII Identity You Can't Fake

A NETWORK WITH NO BOSS has no central authority to vouch for anyone. So how do you know the peer answering you is who it claims to be—and that a message really came from its author? Axona's answer uses cryptography the same way the address scheme uses geography: the guarantee is baked into the identifier itself, checked by everyone, owned by no one.

Recall the node address: `[8-bit region][256-bit hash of public key]`. That second part isn't random—it is a SHA-256 hash of your **public key**.[10] Your key *is* your address. Three things follow, with no server in the loop:

[10] One half of a cryptographic key pair; the other half, the *private* key, never leaves your device.

- **You can't wear someone else's address.** When two peers connect, each must prove it holds the private key matching the address it claims, by signing a fresh one-time challenge tied to that exact connection.[11] Claim an address whose key you don't hold and the signature doesn't check out—the connection is refused. No peer can impersonate another, and none can park itself at a chosen address to intercept a victim's traffic.

[11] Under the hood this is the `axona/4` handshake.

- **Every published message is signed.** A publish carries its author's public key and a signature over the contents; any receiver verifies that signature itself before handing the message to the application. Alter the message and the signature breaks; forge the author and it breaks. Receivers trust the math, not the messenger—even when the message arrived relayed through a dozen strangers.
- **The region is the one thing you *don't* prove**—on purpose. The 8-bit prefix is a hint you choose, your "area code," so you are free to claim any region; but as Idea #1 showed, lying about it only makes your *own* connections slower. Identity is math you can't fake; location is a hint you're free to pick.

What this buys the mission: the network can let *anyone* join with no gatekeeper, no account, no login server—while still guaranteeing

that "who you're talking to" and "who wrote this" are real. No certificate authority, no reputation bureau; just keys and signatures, checked end to end. The cost is small and paid once: proving your key at connect time is a few thousandths of a second of math, and it never touches the speed of routing or delivery afterward.

IX Hitting the Theoretical Floor

In 2004, Frank Dabek and colleagues proved a beautiful, depressing result: **no recursive DHT can be faster than** 3δ, where δ is the median one-way latency between random pairs of nodes on the Internet—on today's internet, roughly 68 thousandths of a second: the time a single message takes to travel one way between two random computers.[12]

[12] Dabek et al., "Designing a DHT for low latency and high throughput," *Proc. NSDI 2004*.

The proof is geometric. The final hop costs a full δ. Each hop *before* that closes half the remaining ID space and, on average, half the remaining geographic distance—so the second-to-last hop costs δ, the third-to-last $\delta/2$, the fourth-to-last $\delta/4$, and so on. The series $\delta + \delta/2 + \delta/4 + \cdots$ sums to 2δ. The total is the final hop plus the prior chain:

$$\delta \;+\; (\delta + \delta/2 + \delta/4 + \cdots) \;=\; 3\delta$$

No matter how clever your routing, no matter how big your network, you can't beat this.

For two decades, no published DHT had been measured at this floor. The best implementations got to maybe $2\times$ the floor.

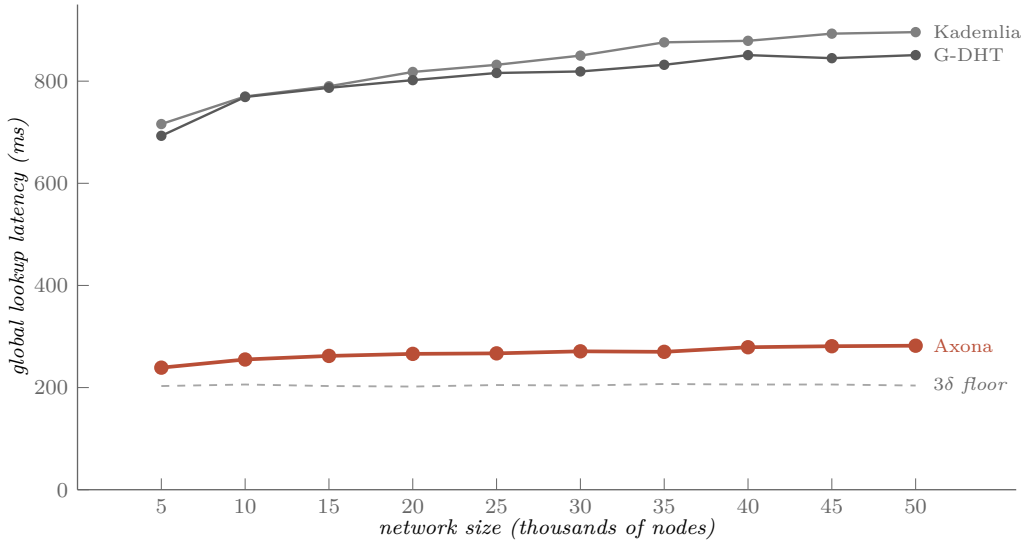


Figure 5: Global lookup latency versus network size at every 5,000 from 5,000 to 50,000 nodes (10× scaling), plotted against the Dabek 3δ analytical floor (gray dashed). Kademlia climbs from 716 to 896 ms as the log-N hop tax compounds; G-DHT tracks Kademlia almost identically on *global* lookups (the geographic prefix only helps regional cells). Axona hugs the floor in a tight 239–282 ms band across the whole

Axona hugs the floor at 1.17–1.38× across a 10× scaling range (5,000 to 50,000 nodes). At 25,000 nodes it routes a global lookup in 267 ms vs a 205 ms 3δ floor ($\delta \approx 68$ ms), and the curve is essentially flat from 15,000 nodes onward: 262, 266, 267, 271, 270, 279, 281, 282 ms as the network grows by another 35,000 peers. The ∼30% residual overhead has a clean structural explanation: Axona takes about 4–5 hops where an ideal protocol would take 3, and each "extra" hop costs about $\delta/2$, exactly as the geometric series predicts.

Plain Kademlia, by contrast, gets *worse* as the network grows: from 716 ms at 5,000 nodes to 896 ms at 50,000. Its log-N hop tax compounds at full per-hop RTT every hop. G-DHT tracks Kademlia almost identically on global lookups—the geographic prefix only helps regional cells, where every hop is short. Axona approaches the floor by learning per-hop locality: its AP scoring weights short-RTT edges so heavily that even a hop "inefficient" by raw count is short in wall-clock time.

The picture changes dramatically when we look at **regional** traffic instead of global. Figure 6 shows the same protocols at the same population sizes, but measured on 2,000-kilometer lookups—queries whose source and target are in the same continental neighborhood.

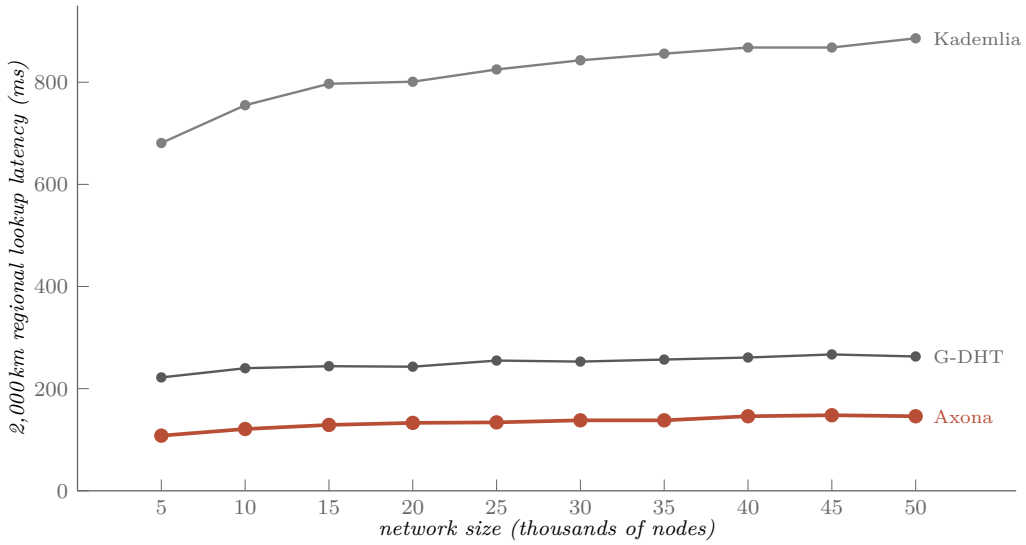


Figure 6: 2,000-kilometer regional lookup latency at every 5,000 from 5,000 to 50,000 nodes. Kademlia climbs from 681 to 886 ms—essentially the same as its global curve, because K-DHT has no way to know the lookup is regional and routes through full-planet hops anyway. G-DHT settles into a 222–267 ms band purely from the geographic prefix: ∼3–3.4× lower than Kademlia, with the curve nearly flat from 10,000 nodes onward. Axona drops further to 108–148 ms by layering learned per-hop locality on top of the structural prefix—roughly 1.8–2× better than G-DHT and 6–6.3× better than Kademlia. All success rates 100%.

The structural payoff of the G-DHT prefix is most visible here. On global lookups (Figure 5), G-DHT tracks Kademlia almost identically because the lookup has to traverse the whole identifier space regardless of how the IDs are arranged. On regional lookups, the prefix is the whole game: a 2,000 km lookup walks IDs that are XOR-close to the source, which under the S2 prefix means they are also geographically close, which means each hop is a short-RTT hop instead of an average-

pair-of-the-globe hop. G-DHT routes a 2,000 km query in 263 ms at 50,000 nodes by paying short-RTT costs on every step.

Axona then adds learning on top. The AP scoring weights short-RTT edges so heavily that the synaptome converges to a structure where almost every reinforced edge is a low-latency one. Regional lookups become a few short hops, and Axona's 108 ms at 5,000 nodes is already comfortably below the 3δ global floor—because the effective δ *within* a continental region is a fraction of the global pairwise median. G-DHT shows what locality buys you structurally; Axona shows what locality buys you when the protocol also learns.

A NOTE ON THESE FIGURES. The per-size sweeps in this section are from the v0.93.0 benchmark, run on the project's earlier "flat-Hilbert" geographic partition. The current build uses Google's standard S2 cells, which shifts regional latencies by a few percent without changing the ordering or the flat-with-scale shape. On the current partition the 25,000-node reference points are **268 ms** for a global lookup and **152 ms** for a 2,000 km regional lookup—still hugging the same floor.

X Is It the Learning, or the Geography?

A SKEPTIC would push back: "Sure, but maybe the geographic prefix is doing all the real work. The brain-inspired learning is gravy."

An **ablation study** answers this. Strip the geographic prefix entirely—random IDs again—and re-run the comparison.

An ablation study removes one feature and re-runs everything to see what that feature actually contributed. It's the cleanest way to attribute a result to a specific mechanism.

Result: with **zero geographic information** at 25,000 nodes, Axona still routes ∼**40% faster than Kademlia** (513 ms vs 852 ms). The learning is doing real work, not sharpening pre-existing geographic structure.

Add the geographic prefix back in and the gap widens to about 68%. Geography helps; geography is not necessary.

This is the kind of clean control experiment that should accompany any claim of this kind.

XI Slice World

THE SHARPEST demonstration is the **Slice World** test. The network gets cut almost in half—Eastern hemisphere on one side, Western on the other, connected only through a *single node* near Hawaii. Every other cross-hemisphere connection is severed.

Can the protocol still route messages between hemispheres?

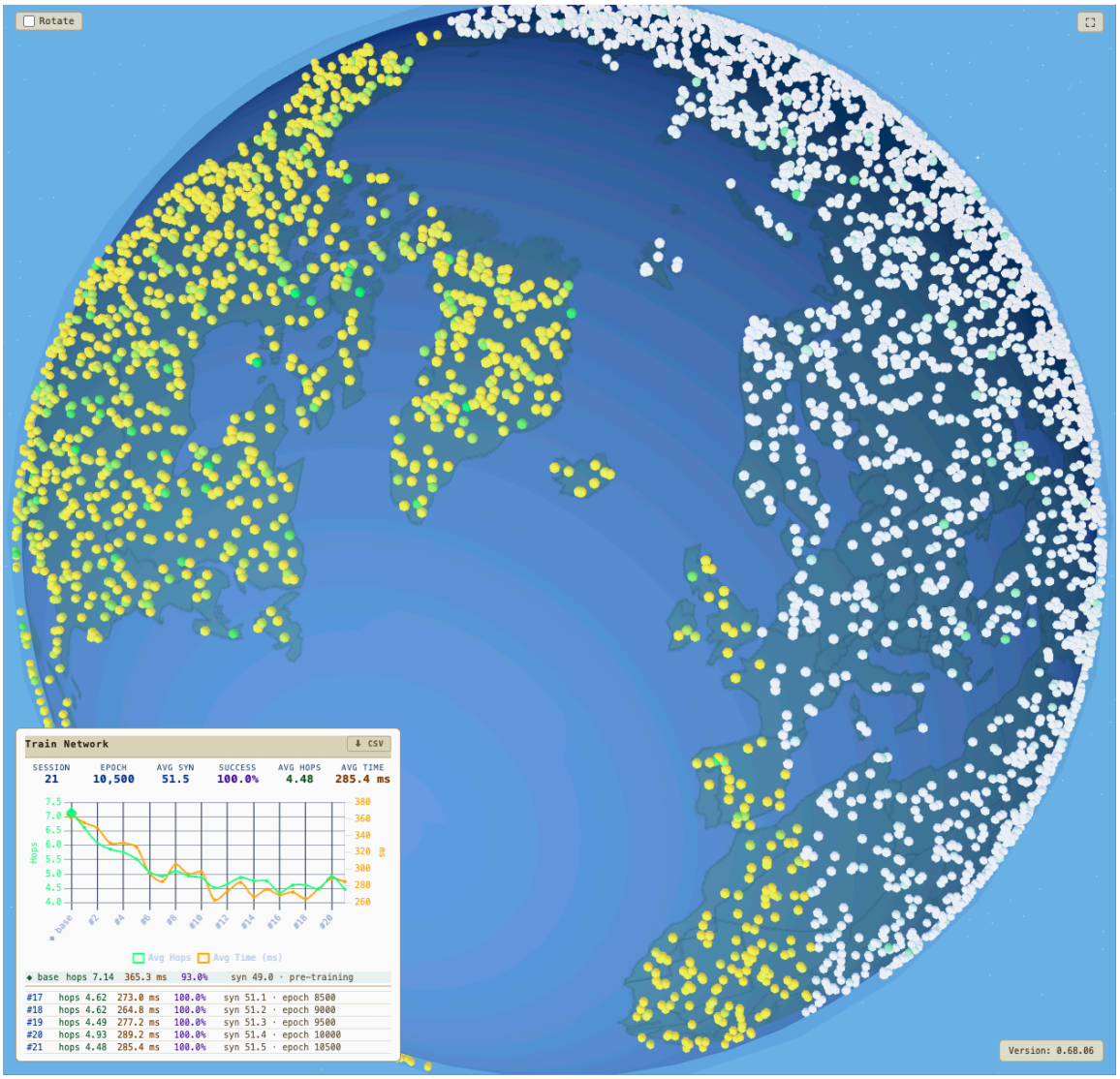


Figure 7: Slice World, the single-bridge partition test. The simulator removes every cross-hemisphere edge except those incident on one node near Hawaii. The question is not whether the protocol can find the bridge (it's a one-shot routing problem) but whether repeated successful crossings dissolve the partition.

- **Plain Kademlia: 0% success.** With no learning, the partition is permanent.
- **G-DHT (geography only, no learning): 4.6% success.** The geographic prefix accidentally points a few peers at the bridge, but nothing builds on the discovery.
- **Axona: ∼94% success.**

The bridge becomes a **seed crystal**. After just 10 lookups through the partition, hop caching has installed cross-hemisphere edges in many intermediate nodes. Triadic closure creates direct connections between peers that keep meeting through the bridge. By 500 lookups,

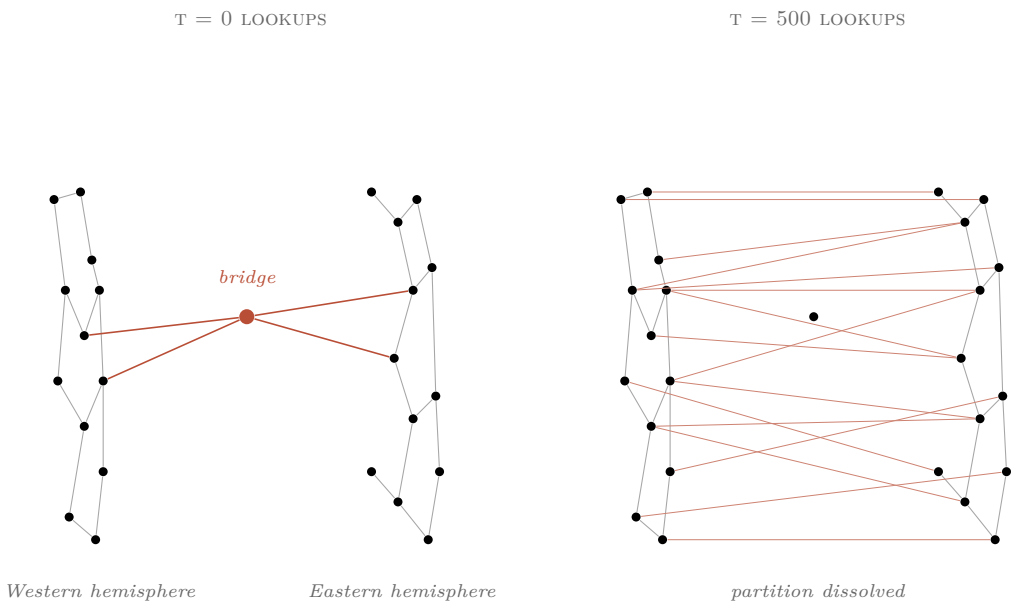


Figure 8: At $t = 0$ (left), the partition has a single bridge node connecting two hemispheres. By $t = 500$ lookups (right), hop caching has installed cross-hemisphere shortcuts in many intermediate nodes; triadic closure has converted observed transit pairs into direct edges. The bridge is no longer special. The protocol does not keep finding the bridge—it uses the bridge to rebuild the bridges that were cut.

hundreds of cross-hemisphere connections exist. The partition has effectively dissolved.

The protocol doesn't keep finding the bridge—it *uses* the bridge to rebuild the bridges that were cut. Like a brain forming new pathways around damaged tissue.

XII How Does This Become Real Software?

The simulator is the lab—fifty thousand simulated peers in a single browser tab, no real network underneath. The same code has to eventually run on the actual internet, where messages take real milliseconds and connections occasionally die. How do you get there?

Axona's architecture stacks into **three layers**, with a contract at each interface (Figure XII). Everything above the top contract is *the application*; everything between the two contracts is *the protocol*; everything below the bottom contract is *the network*. The contracts are deliberately rigid—neither side is allowed to reach across—which is what makes the simulator's numbers transfer to real deployment.

At the top is the **application layer**—the code that wants to do something with a peer-to-peer network. A chat client, an incident-reporting map, an agent-collaboration backend. The application layer doesn't care how routing works internally; it just wants to *do things*. What it sees is the **DHT contract**—the `AxonaPeer` surface[13] from Section VII, organised into five clusters: **lifecycle** (`start`, `stop`,

[13] `AxonaPeer` and the `Transport` abstract base both live in the `@axona/protocol` kernel package; the routing logic and learning rules sit on top of `AxonaPeer` and call through `Transport`.

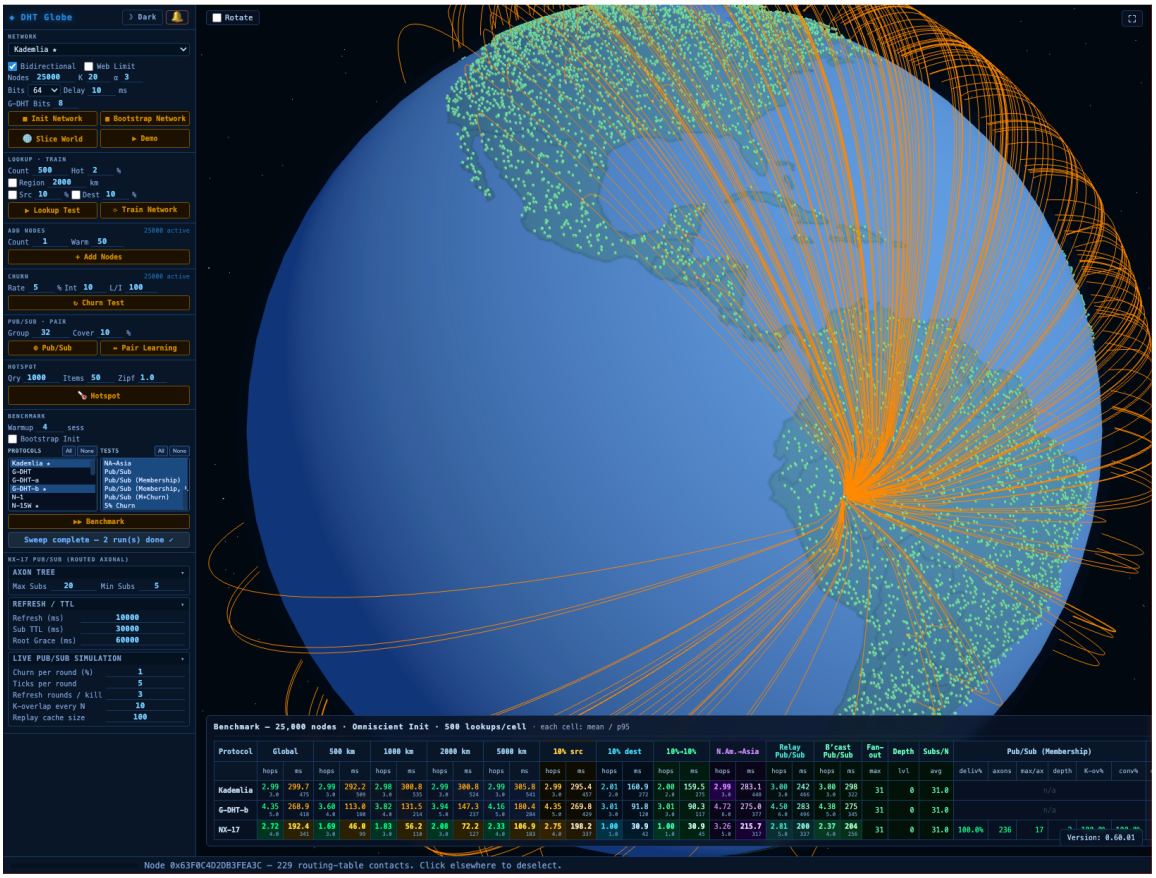


Figure 9: The simulator at 25,000 peers on a 3D globe. Every dot is a node; edges are synapses (peer-to-peer routing connections). The visualization runs in a single browser tab—the same JavaScript that ships in the production peer. This is what "the simulator is the protocol" looks like.

`join`, `leave`); **pub/sub** (`pub`, `sub`, `pull`, `metrics`); **direct messaging** (`send`, `notify`, `onMessage`); **mesh introspection** (`peers`, `onPeerJoin`, `onPeerLeave`, `lookup`); and **telemetry** (`health`, `onLog`, `onError`)—a way for the application to *watch* what the protocol is doing without being able to mess with it.

In the middle is the **protocol layer**—the routing decisions. This is where Axona's actual algorithms live: AP scoring, hop caching, vitality function, axonal trees, all the brain-inspired machinery. The same slot can hold a different routing protocol entirely—plain Kademlia (K-DHT) or geographic-prefix Kademlia (G-DHT)—and the layers above and below don't notice. That interchangeability is what made the benchmark grid possible: each candidate protocol drops into the same slot and runs against the same application code and the same Transport, so the numbers compare directly.

At the bottom is the **transport layer**—the actual machinery that moves bytes between machines. What the protocol layer sees of it is the **Transport contract**: open a channel to a peer, close a channel, send a message and wait for the reply, send a message and don't wait, register a callback for when a peer dies, ask for a peer's measured

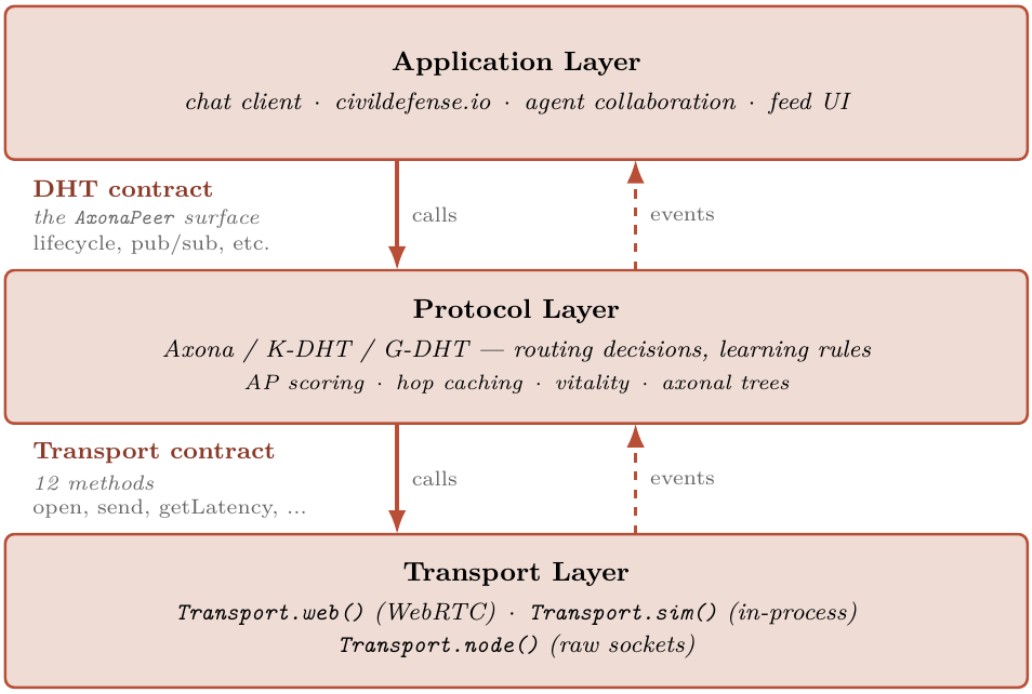


Figure 10. The three-layer architecture. The application layer (chat clients, civildefense.io, agent-collaboration backends) sits above the *DHT contract*—the `AxonaPeer` surface, organised into lifecycle, pub/sub, direct-messaging, mesh-introspection, and telemetry clusters. Below the DHT contract lives the *protocol layer*: the routing decisions and learning rules—Axona, or K-DHT, or G-DHT. Below *that* is the *Transport contract*, a twelve-method surface that the network has to provide. Concrete Transports plug in underneath: `simTransport()` for the in-browser lab, `webTransport()` for WebRTC in real browsers, `nodeTransport.server()` for headless servers. Calls travel downward; events travel upward. Neither contract permits the layers on either side to know what the other is doing internally.

latency. Twelve methods. That's it.

Three concrete Transports plug into the same contract. `simTransport()` runs the network in-process inside a single browser tab—this is what produces the 25,000-peer benchmark numbers from earlier sections. `webTransport()` runs WebRTC data channels between real browsers. `nodeTransport.server()` runs raw sockets on headless servers. The protocol layer calls *down* into whichever Transport is plugged in ("send this peer a message asking for its closest synapses to target X") and emits events *up* through the DHT contract ("a lookup just completed; here's the result"). It doesn't know which Transport is underneath, and **it can't tell the difference**, by design.

That last point is what matters. When the simulator says "Axona takes about 5 hops on average to find a target in a 25,000-node network," that number isn't a simulator artifact. It's a property of the protocol code, which is the same code that will run when this gets deployed. The Transport changes; the protocol doesn't. The simulator's hop counts, latency curves, churn-resilience numbers—they all transfer

to the real internet because the routing decisions that produce those numbers are made in code that doesn't know it's being simulated.

So the simulator is the deployment vehicle, and the plumbing on the other side now exists. The production Transport—a browser-resident node called **axona-peer**, running on WebRTC data channels—is live at `https://axona.net`; a minimal self-contained reference peer—the same kernel, a few hundred lines of application code—runs at `https://demo.axona.net`. A signaling broker, **axona-bridge**, runs at `https://bridge.axona.net` and handles the WebRTC offer/answer exchange that two peers behind NATs need to find each other; it carries no application traffic and any operator can stand one up, so the rendezvous role is interchangeable. The cold-start problem—finding your first peer when you've never been on the network before—resolves through any of three `BootstrapEndpoint` variants: a rendezvous URL with a signed manifest, a QR-code-pasted pairing string for direct device-to-device pairing, or an in-process simulator pointer. Once bootstrap returns one open channel, the routing logic is unchanged—the same `lookup_step` chain that the simulator runs.

The first end-user application running on Axona in production is `civildefense.io`, a tap-to-report incident map built in weeks because the substrate primitives (signed posts, geographic locality, 24-hour expiry, anonymous P2P) inherit directly from this protocol layer. Source for the three live components: `github.com/axona-net/axona-peer`, `github.com/axona-net/axona-bridge`, `github.com/axona-net/dht-sim`.

XIII What the Simulator Doesn't Model

It is worth being explicit about what *isn't* measured.

The simulator runs about 25,000 lines of JavaScript code modeling the network, but it abstracts away several real-world frictions:

- **Connection setup time.** Real WebRTC connections take 1.5–3 seconds to negotiate. The simulator treats them as instant, so real-world recovery from partitions will be slower than the simulator suggests.
- **Timeout windows.** Real RPCs to dead nodes stall for seconds before failing. The simulator detects death instantly.
- **Bandwidth saturation.** Initially feared as a *success-disaster* failure mode for adaptive routing—that AP scoring's preference for fast peers might funnel traffic onto a few overloaded nodes, oscillating as they collapse and recover. Measurements of per-node traffic

distribution at every tested scale (5,000 to 50,000 nodes) show the opposite: Axona distributes load broadly across the population while plain Kademlia and G-DHT concentrate it. At 50,000 nodes, *zero* Axona nodes process more than 100× the network mean traffic; Kademlia produces 56 such nodes, G-DHT produces 62. Bandwidth concentration is much less of an issue for Axona than for Kademlia—not a non-issue, but not the deploy blocker first feared.

- **Latency jitter.** Real round-trip times vary by ±30% from queuing and congestion. The simulator's latencies are clean and monotone.

These get called out in a separate red-team analysis.[14] The protocol's measured results show **the brain working perfectly**; the *body*—actually deploying it on real internet conditions—still needs work.

[14] The companion red-team analysis ranks thirteen deployment-relevant gaps by severity and time-to-fix.

XIV Plasticity of Plasticity

The most interesting future direction is **metaplasticity**—plasticity of plasticity. In real brains, the rules governing learning *themselves* change based on the brain's activity level. A neuron that's been very active becomes harder to strengthen further (otherwise everything saturates). A neuron that's been quiet becomes easier. The learning rules adapt.

Axona's parameters—decay rate, protection window, exploration rate—are currently hand-picked constants. A metaplastic version would let the network self-tune them based on local conditions. A peer in a stable region would adapt slowly. A peer in a high-churn region would adapt aggressively. The user would set one knob—"I want my lookup failure rate below 1%"—and the network would tune itself to hit it.

That's the next layer of brain-inspired self-organization, and the natural endpoint of the path the protocol lays out.

XV Why This Matters

Step back from the technical details. The big idea:

A peer-to-peer network and a brain face the same engineering problem—limited connection capacity, vastly more candidates than slots, and the need to figure out which connections are useful.

The brain's hundreds-of-millions-of-years-old solution maps directly onto network routing. Strengthen what works. Decay what doesn't. Protect recent learning briefly so it isn't immediately overwritten. Occasionally explore. The result is a network whose structure is shaped by the traffic it carries, rather than by rules an engineer guessed at.

This isn't just a faster DHT. It's a demonstration that adaptive systems with the right learning rules find their own structure—that ten well-chosen rules grounded in biology can deliver what twenty years of DHT engineering had not, and that you can hit a theoretical limit that's stood for two decades by importing the right idea from a different field.

Code, data, simulator, and the red-team analysis criticizing it are open source.

Postscript: Symbiotic Development

As you may have guessed, this work heavily relied on AI—Claude Code Opus 4.7, to be specific. Though I am confident I could have designed and built this system without that help, I also know I could not have done it as quickly or as robustly as I did here. Virtually every design decision and algorithm was mine alone, or based upon existing systems and processes, with guidance and criticism on the actual coding. That said, this has been an incredibly symbiotic experience. Claude acted not as a passive implementor but as an *idea amplifier*. In fact, the AI played many roles—application and system coder, graduate student performing the research activities, advisor on technical questions, critic on design directions, co-author (usually main author) of the documents—a true collaborator enabling us to jointly create something that had only existed, in a vague form, in my mind and in my notes. Like a few programmers, I have the ability to see the "machine" I wish to create in my head, and even operate it there to a degree; but transferring that working virtual system into reality is always a challenge. I have always looked at code as the unimportant-but-necessary part of the creation process.

I should make clear that my own expertise in code was essential to this task. **This is not a "vibe-coding" project.** There is a meta-level of functionality that we worked to bring to life, but the actual functionality is determined by the implementation and the details of that implementation. With a system like this, built inside a simulation of the world, it is extremely easy to design things in ways that take advantage of the architecture of the simulator itself—information and access that is simply unavailable in real life. That leads to extraordinary performance increases that cannot survive reality when it shows up. AI coders work to optimise the system in front of them and have no real conception of the boundaries between systems; they are extremely good at leveraging the gaps between those boundaries to achieve a particular goal. This behaviour was a huge challenge, because like any human I had the desire to see the magic happen. I was a believer, and the AI would regularly reward my belief in how amazing the system would be. But that magic almost always proved to be a mirage.

Luckily, I was well equipped to address this. I am a particularly good programmer and system architect, but oddly, my greatest talent may be that I am an extraordinarily good debugger. **Great debugging is based upon one thing—scepticism.** Every new line of code you write probably has a bug. Every algorithm has a side effect you can't see. Every system has invisible holes just waiting for your

application to step into them and die. The current system, I have no doubt, has countless numbers of these, and they will certainly become readily visible as we deploy. Reality is an unforgiving bitch.

In perhaps the same way the three-layer Axona system works—application / protocol / transport—the process here was *concept and design (Smith) / coding and documentation (Claude) / criticism, quality assurance, and debugging (Smith).* This iterative loop led to a new use for an old mantra:

If it is too good to be true, then it is not true.

The development of the Axona system was not magical—it was very complex, very tedious, and frankly more than a little scary. When I discovered that the system had "cheated" in its performance—which happened numerous times—each instance could have meant the entire architecture was a house of cards that would never stand in reality. Usually these cheats were applied uniformly across all protocols, so that when the fix landed every protocol now showed more reliable numbers, and the *ratios* of performance between them held constant. That repeated pattern is what kept the comparative claims in this paper alive even as the absolute numbers shifted underneath them.

This development required numerous systems and subsystems just to provide a laboratory within which the design could be explored. That laboratory was easily the most important part of the process.

Most simulation work in distributed systems begins with a protocol and writes a testbed to validate it. We did the opposite. We built the simulator first—about 25,000 lines of JavaScript that creates N peers on a virtual globe, runs them through a fixed benchmark grid, and emits one CSV per run. Then we used that lab to design the protocol.

Step 1 — The simulator before the protocol

The first commit to `dht-sim` is dated **March 19, 2026**. The first benchmark CSV is dated March 24. From that point forward, every protocol idea had to survive the grid before earning the right to be carried forward to the next iteration.

The grid was deliberately fixed at the start and held stable for the rest of the run. Ten population sizes—5,000 nodes stepping by 5,000 up to 50,000. Five test cells per population—global lookups, regional 500 km / 2000 km / 5000 km, and 5% per-tick churn. A separate single-bridge partition test ("Slice World") that asks whether the protocol can dissolve a network split by repeated successful crossings. Three metrics per cell—hop count, wall-clock latency, success rate. The simulator runs the whole grid in a single browser tab; the results emit as one CSV per run, archived in `dht-sim/results/`.

The point of fixing the grid early was not measurement aesthetics. It was a methodological commitment: every later protocol idea had to be judged against the same yardstick so that promotion and retirement were measurable, not editorial. The grid is the lab's calibration. If it drifts, the lab stops working.

Step 2 — Build the baselines: K-DHT and G-DHT

Before measuring any learning-adaptive protocol, we needed something to measure against. Two classical DHT designs went in first, both implemented as real runnable code paths inside the simulator and both running the same benchmark grid every later protocol would be judged against.

K-DHT is plain Kademlia: random node identifiers in the same 264-bit address space all three protocols share, XOR distance, $K = 20$ buckets, $\alpha = 3$ parallel queries. No locality, no learning. It is the dominant deployed DHT in production—BitTorrent, IPFS, Ethereum's peer-discovery layer, and Tor v3 hidden services all run Kademlia variants—and therefore the baseline every new design has to beat to justify itself.

G-DHT is K-DHT with one structural change: the top 8 bits of every node identifier encode the S2 cell of its claimed position (the construction is in §"Putting the Address in the Address"). XOR distance now approximately tracks physical distance for free; the routing algorithm is unchanged. G-DHT tests an important boundary question—how much of the lookup-latency win comes from *structure* and how much from *learning*—and is also the substrate Axona's learning is eventually layered onto in the deployed protocol.

Both were running against the grid by early April. Their CSV rows became the reference lines every subsequent protocol's measurements would be compared against. The "Axona vs. Kademlia" claims you see throughout this explainer are the residue of forty-five later protocols measured against these two baselines.

Step 3 — Iterate the neuromorphic family against the grid

With the lab built and the baselines fixed, we started designing learning-adaptive protocols. The first was **N-1** in late March—classical neuromorphic ideas applied to routing: Hebbian long-term potentiation, a routing-table population of "synapses" with adaptive weights, time-based decay. We ran it against the grid. We learned which cells it won and which it lost. We then designed N-2, ran it, kept the parts that improved cells, and threw out the parts that didn't. Then N-3. And so on.

Nine weeks later, by mid-May, we had measured **47 distinct DHT**

designs (counting the two baselines, the eventual deployed protocol, and 44 retired variants in between):

Family	Count	Members
Kademlia (baseline)	1	K-DHT
G-DHT (geographic prefix)	4 hist. → 1	G-DHT-8, G-DHT-10, G-DHT-a, **G-DHT** (surviving `geob` variant)
N-series (first neuromorphic)	17	N-1, N-2, N-2-BP, N-2-SHC, N-3, N-4, N-5, N-5W, N-6W, N-7W, N-8W, N-9W, N-10W, N-11W, N-12W, N-13W, N-15W
NS-series (intermediate)	6	NS-1 …NS-6
NX-series (focused iteration)	17	NX-1W, NX-2W, NX-3 …NX-17
NH-series (consolidation)	1	NH-1
Axona (deployed)	1	Axona
Total distinct protocols	**47**	—

The N-series was the original exploration. Branches with suffixes BP (back-propagation), SHC (synaptic-hop cache), and W (weighted) carried experiments that diverged from the trunk and were rolled back when they didn't pay off. NS-1…NS-6 was an intermediate generation focused on the synaptome data structure itself. NX-1…NX-17 was the focused iteration—each NX adding or testing a single mechanism, measuring it against the grid, keeping or retiring. NX-17 was the high-water mark by mechanism count: 18 distinct rules and 44 parameters.

Step 4 — What promotion and retirement looked like

Almost every retirement was the same shape. A candidate mechanism made one cell better and another cell worse. NX-11 added a "second-tier" synaptome scoring—r500 improved by 8 ms, churn-cell success dropped from 99% to 96%. NX-12 tried to fix the churn cell by lengthening the protection window—r500 regressed back. The grid made the trade visible immediately. Most NX revisions died this way: not because they were bad ideas, but because they couldn't simultaneously satisfy seven cells.

Promotion looked different. A mechanism would win on its native cell and survive into the next generation even when the protocol around it was retired. **Long-term potentiation** came forward intact from N-5W and has lived in every neuromorphic protocol since. **Surrogate routing** (the "iterative fallback") came from NX-4; every protocol without it failed Slice World at 0%, so every protocol from NX-4 onward kept it. **Hop caching** came from NX-6 because it short-

ened paths the next iteration could measure. **Triadic closure** came from NX-7. **Two-hop lookahead** came from NX-15. By the time NX-17 consolidated everything that had survived, the rule set was 18 deep—and every one of those 18 had a falsification trail across earlier iterations.

NH-1 then collapsed NX-17's 18 rules into 12, and 44 parameters into 12, by keeping only the rules with the deepest falsification trails and folding the rest into a single vitality function. NH-1 is not a design choice; it is what was left after the things that didn't work were measurably removed. That is what Axona's routing logic is.

A few dead ends are worth noting because they're informative. NX-3 tried routing without iterative fallback; Slice World success collapsed to 0% and the test caught it immediately. NX-14 lost to NX-15 on r500 by 12 ms and the source file is now retired from the tree (but the CSV is still in `results/`). NX-16 tried to fix routing-table imbalance by masking the low bits of the distance metric; global latency exploded and the protocol now lives in `axona-docs/dead-ends/` as a cautionary example.

Step 5 — Unbinding the protocol from the simulator

By early May we had NH-1—twelve rules, twelve parameters, one vitality function, measurably the strongest variant. But at this point the protocol was still defined by its simulator. The code ran in a single Node.js process where every node had direct access to every other node's state through a god's-eye `nodeMap`. That was useful for development and unacceptable for deployment. A real peer-to-peer system has no god's-eye view; every peer only knows what it learns through actual network messages.

So we did something most simulator-first projects never do. We refactored the protocol code until the simulator's view collapsed into the same view a real network would have. Over roughly fifteen commits, every cross-peer read became a `transport.send()` call. Every liveness check became a real heartbeat. Every routing decision was made by the peer that owns the data, not by the source of the lookup. The `nodeMap` survived only in places that orchestrate the simulator itself—spinning a node up, tearing it down—which production replaces with operating-system process startup and shutdown.

We then ran a parity-gate benchmark: 25,000 nodes, before and after the refactor. The neuromorphic protocols came out within 5% on hop counts and 1% on latency. The small drift upward was the architecturally-honest cost of letting each peer make its own decisions instead of the source orchestrating the walk. K-DHT, G-DHT, and the prior NX variants came out within 1% across the board.

After that refactor, the simulator stopped being a simulation of the protocol. It became the protocol. The same `AxonaPeer.js` that runs inside the 25,000-node browser-tab simulation is the `AxonaPeer.js` that ships in `axona-peer` and runs on `axona.net` over real WebRTC connections. The Transport layer changes—`SimulatedTransport` in the simulator, `WebRTCTransport` in production—but the routing logic is identical, byte-for-byte. Every hop count, every latency curve, every churn-resilience number from the simulator transfers to the real network because the routing decisions that produce those numbers are made in code that doesn't know it's being simulated.

This is the part of the project I would put on the table first if I were arguing for it intellectually. Not the 1.33× floor number. Not the 47-protocols-in-9-weeks pace. **The fact that the lab and the deployed system are the same code.**

Step 6 — The fixed-grid claim, in retrospect

All of the above only works because the benchmark grid stayed stable as the protocol churned. The cells the simulator iterates against have been essentially unchanged since week one. The latency model got a small refinement in v1.1.2 (live per-hop RTT instead of a value stamped at synapse admission). The churn-rate model got more aggressive somewhere around NX-11. But the cells are the same. A CSV from April 8, 2026 reading "NX-7 global 5K = 243.6 ms / 3.28 hops / 100%" is directly comparable to today's CSV reading "Axona global 5K = 238.75 ms / 4.33 hops / 100%"—six weeks apart, same grid, similar wall-clock at different hop counts (Axona uses learned RTT routing where NX-7 used pure XOR distance, which is why Axona walks more hops but each one is shorter).

That comparability is what lets the simulator be a falsification tool rather than a proof-of-concept tool. It is what lets us say "47 variants were measured" rather than "47 variants were imagined."

It is also what made the v0.93.0 `super.buildRoutingTables` discovery from earlier this month into a methodological vindication rather than a methodological embarrassment. A bilateral-cap bug had been silently inflating Axona's apparent advantage for some unknown number of measurement cycles. Once we found it, we re-applied the grid with the fix in place, and the conclusion held—Axona ties the underlying NH-1 / NX-17 routing kernels on routing quality, and wins decisively on churn. Without the grid, that correction would have been a self-report. With the grid, every prior measurement that touched the bug became re-runnable and the residue became *more* defensible, not less.

What it adds up to

In a normal research paper you see the final design. You can't tell whether the 12 rules of NH-1 were chosen because they work or because the author happens to like them. Here every rule has a falsification trail. The lab is open source. Every CSV is in the repository. Every retired protocol's measurements are still on disk. The simulator is the protocol, and the protocol is the deployed network.

The clean 3-way comparison in the paper is not the design. It is the fossil record. The design is everything that is not in the paper anymore—and the simulator that produced it and the production network running it are the same code.